

Node.js + JavaScript — Day 1 Quick Revision

A compact pre-Day-2 reading sheet based on the topics covered in our Node.js journey and today's discussion.

1. JavaScript Basics

JavaScript is a programming language. It needs an engine/runtime to execute. Variables store values or references, and functions group reusable logic.

Arrays: ordered collections such as `[10, 20, 30]`. Important basics include indexing, `length`, looping, and array methods.

for loop: repeats code while its condition remains true.

```
for (let i = 0; i < arr.length; i++) { console.log(arr[i]); }
```

Objects: key-value data such as `{ name: 'shirt', price: 799 }`.

User input: in browser JS, input commonly comes from HTML elements/forms. JS reads the value and reacts to events such as clicks or Enter.

2. Browser JavaScript, V8 and Node.js

V8 is Google's JavaScript engine. Chrome uses V8 to execute JavaScript.

Node.js is a runtime environment that lets JavaScript run outside the browser. Node.js uses V8, but Node.js and V8 are not the same thing.

```
Chrome → V8 → executes browser JavaScript
```

```
Node.js → V8 + Node APIs → executes server-side JavaScript
```

Key distinction: JavaScript = language; V8 = engine; Node.js = runtime environment.

Node provides capabilities around V8, including APIs for files, networking, processes, environment variables, and other server-side work.

3. Frontend → Backend → Database

Example: you search for **shirt** on an e-commerce site.

Stage	What happens
Frontend	Reads the search input and sends an HTTP request.
Request	Example: <code>GET /api/products?search=shirt</code>
Backend	Receives the request, applies logic, and asks a data source for matching products.
Database	Stores product data and returns matching records to the backend.
Response	Backend commonly sends the result as JSON.
Frontend	Receives JSON and renders the products on screen.

Important: backend and database are separate. The backend is the middle layer that receives requests, performs business logic, accesses databases/services, and sends responses.

4. Express + HTTP Basics

HTTP methods: GET → retrieve, POST → create/send, PUT/PATCH → update, DELETE → remove.

```
app.get('/products', (req, res) => { ... })
```

req contains information from the client; **res** is used to send the server's response.

Route parameters: `/users/:id` → `req.params.id`.

Query parameters: /products?search=shirt → req.query.search.

Request body: JSON/data sent with POST, PUT or PATCH can be accessed through req.body when body-parsing middleware is configured.

Status codes: 200 OK, 201 Created, 400 Bad Request, 401 Unauthorized, 404 Not Found, 500 Server Error.

JSON is a common format for API request and response data.

5. fetch + Promises + async/await

fetch() lets frontend JavaScript make HTTP requests.

```
const response = await fetch('/api/products?search=shirt');
const data = await response.json();
```

Promise: represents a result that becomes available later. **async** makes a function return a Promise. **await** waits for a Promise inside an async function.

In backend code, async/await is especially common for database calls and other asynchronous operations.

6. Postman

Postman is an API testing tool. It lets you send HTTP requests directly to your backend without needing a finished frontend.

```
Postman → GET http://localhost:3000/products → Node/Express → response
```

You can test methods, query parameters, headers, JSON bodies, authentication, and status codes.

7. Important JavaScript Topics for the Journey

Already encountered: variables/references, functions, arrays, objects, loops, user input, fetch, JSON, async/await, frontend-backend request flow.

Important next topics: error handling, Promises in more depth, destructuring, spread/rest, modules, npm, package.json, environment variables, and Express middleware.

8. Quick Mental Model

Concept	Remember
JavaScript	The language.
V8	Engine that executes JavaScript.
Node.js	Runtime that uses V8 and adds Node APIs.
Express	Node.js web framework used to build servers/APIs.
Frontend	Browser-side code/UI and user interaction.
Backend	Server-side logic, APIs, authentication, data access, etc.
Database	Persistent storage queried by the backend.
Postman	Tool for manually testing APIs.

Core architecture: Browser/frontend → HTTP request → Node/Express backend → database/services → HTTP response → frontend renders the result.

Study rule for your next sessions: you ask the questions; we follow your path. This PDF is only a quick refresher, not a fixed curriculum.